INSTITUTO NTA
Formação em Engenharia de Dados

Python para Engenharia de Dados

Parte 1: Fundamentos da linguagem

APOSTILA DO ALUNO | PARTE 1 DE 2

Conteúdo

- Programação e lógica
- Python como idioma
- IDE e VS Code
- Características do Python
- Variáveis, tipos de dados e operadores
- Coleções: list, tuple, dict e set
- Mini projeto: relatório de vendas
- Erros e try/except

Responsável técnico: Thiago Azeredo Rodrigues

Sumário

1. Programação e lógica 3

2. Python é como um idioma 4

3. IDE e VS Code 5

4. Características do Python 7

5. Variáveis 11

6. Tipos de dados 13

7. Operadores 18

8. Coleções: list, tuple, dict e set 19

9. Mini projeto: relatório de vendas 22

10. Erros e tratamento de erros 24

11. Exercícios 28

12. Resumo e próximos passos 28

Gabarito dos exercícios 30

Glossário 37

Como usar esta apostila

Esta apostila acompanha a **Parte 1** do curso Python para Engenharia de Dados. Ela é o material de estudo: traz a teoria completa, exemplos com o resultado de cada código, exercícios e gabarito. Na aula, o foco é executar o código ao vivo. Aqui você revisa e aprofunda no seu ritmo.

Material	Para quê
Apostila (este PDF)	Estudo e consulta: teoria, exemplos com saída, exercícios e gabarito
Notebook `.ipynb`	Prática: todo o código da apostila pronto para executar e alterar

> **Antes de começar.** Você precisa de Python 3.11 ou superior e do VS Code (o Módulo 3 mostra a instalação). Alternativa sem instalar nada: abra o notebook no Google Colab.

1. Programação e lógica

Um computador executa instruções com enorme velocidade e precisão, mas não tem iniciativa: faz exatamente o que mandamos, na ordem em que mandamos. **Programar é escrever essas instruções de um jeito que a máquina consiga executar.**

1.1 O que é uma linguagem de programação

Imagine que você precisa dar instruções a alguém que só entende um idioma muito específico, com regras rígidas e nenhuma margem para interpretação. Uma **linguagem de programação** é esse idioma: um conjunto de palavras e regras que permite escrever instruções que o computador entende.

O processador (o "cérebro" do computador) só entende **código de máquina**, que são sequências de zeros e uns. Ninguém escreve zeros e uns à mão. Por isso foram criadas linguagens mais próximas do idioma humano, e um programa **tradutor** converte o que escrevemos para código de máquina. Existem dois tipos de tradutor, o compilador e o interpretador, que explicamos no Módulo 4.

1.2 Lógica e algoritmo

Antes de qualquer linguagem vem a lógica. Um **algoritmo** é uma sequência finita e ordenada de passos para resolver um problema. Pense numa receita de bolo: os ingredientes são a **entrada**, o modo de preparo é o **processamento** e o bolo pronto é a **saída**. Todo algoritmo tem essas três partes.

Exemplo: calcular a média das notas de um aluno. Em português, os passos são:

```
1. Receber as notas do aluno          (entrada)
2. Somar as notas                      (processamento)
3. Dividir a soma pela quantidade      (processamento)
4. Mostrar a média                     (saída)
```

Agora o mesmo algoritmo em Python, em três células, um passo de cada vez.

Passo 1: guardar as notas. A linha abaixo cria uma **lista**, que é um conjunto de valores entre colchetes separados por vírgula. O nome `notas` passa a se referir a essa lista. Atenção: os números decimais usam **ponto**, e não vírgula.

```
notas = [7.5, 8.0, 6.5]
```

Passo 2: calcular a média. `sum(notas)` soma todos os valores da lista (7,5 + 8,0 + 6,5 = 22,0). `len(notas)` conta quantos itens a lista tem (3). A barra `/` faz a divisão. O resultado fica guardado no nome `media`.

```
media = sum(notas) / len(notas)
```

Passo 3: mostrar o resultado. `print(...)` exibe algo na tela. O `f` antes das aspas cria uma *f-string*: o que estiver entre chaves `{ }` é trocado pelo valor. O trecho `:.2f` pede duas casas decimais.

```
print(f"Média: {media:.2f}")
```

SAÍDA

```
Média: 7.33
```

> **Ideia central.** Toda lógica de programação se resume a três estruturas: **sequência** (um passo depois do outro), **decisão** (se isso, então aquilo) e **repetição** (faça várias vezes). Nesta Parte 1 trabalhamos a sequência e os dados. Decisão e repetição entram na Parte 2.

Por que lógica primeiro? Porque a sintaxe muda de linguagem para linguagem, mas a lógica permanece. Quem pensa bem o problema escreve o código em qualquer linguagem.

2. Python é como um idioma

A melhor forma de aprender Python é tratá-lo como um idioma novo. Você precisa de vocabulário, gramática e prática. A analogia é bem próxima:

Idioma	Python
Vocabulário	Palavras reservadas (`if`, `for`, `def`) e funções prontas (`print`, `len`)
Gramática	Sintaxe: dois-pontos, indentação, parênteses
Pontuação	Aspas, vírgulas, colchetes `[ ]` e chaves `{ }`
Frases	Instruções (comandos) e expressões (cálculos que produzem valor)
Textos	Scripts (`.py`) e notebooks (`.ipynb`)
Dicionário	Documentação oficial e a função `help()`
Normas de escrita	O guia de estilo PEP 8
Comunidade de falantes	Bibliotecas e ecossistema (pandas, PySpark, SQLAlchemy)

2.1 A primeira frase no novo idioma

Uma **função** é um pequeno programa pronto que executa uma tarefa. `print` é a função que escreve algo na tela. Os parênteses recebem aquilo que a função deve usar, e um texto sempre fica entre aspas. Execute a célula:

```
print("Olá, engenharia de dados!")
```

SAÍDA

```
Olá, engenharia de dados!
```

2.2 Comentários

Tudo que vem depois do símbolo `#` é ignorado pelo Python. Os comentários servem para deixar anotações para humanos que vão ler o código, inclusive o você do futuro.

```
# esta linha é só um comentário, o Python ignora
print("Esta linha executa")  # comentário também pode ficar no fim da linha
```

SAÍDA

```
Esta linha executa
```

2.3 O vocabulário reservado

Algumas palavras têm significado fixo para o Python, como `if`, `for` e `class`. São as **palavras reservadas** e não podem ser usadas como nome de nada. O comando `import` traz para o seu código uma "caixa de ferramentas" pronta; aqui trazemos a caixa `keyword`, que sabe listar essas palavras. O trecho `[:8]` pede apenas as 8 primeiras (você verá esse recurso, chamado fatiamento, no Módulo 6).

```
import keyword
print(keyword.kwlist[:8])
```

SAÍDA

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await']
```

Gramática. Em Python, a **indentação faz parte da sintaxe**. Onde outras linguagens usam chaves `{ }` para delimitar blocos, Python usa o recuo (4 espaços por convenção). Um recuo errado gera `IndentationError`.

3. IDE e VS Code

3.1 O que é uma IDE

IDE significa **Ambiente de Desenvolvimento Integrado**. É um programa que reúne, num só lugar, o editor onde você escreve o código, a execução, o depurador (que ajuda a encontrar erros), o terminal e o controle de versão. Dá para programar com um bloco de notas e um terminal, mas a IDE oferece destaque de cores na sintaxe, autocompletar, aviso de erros enquanto você digita e execução passo a passo.

Ferramenta	O que é	Quando usar
VS Code	Editor leve e extensível, gratuito. Vira IDE com extensões	Scripts, projetos, notebooks, pipelines. Nossa escolha
PyCharm	IDE completa focada em Python (versão Community gratuita)	Projetos grandes só em Python
Jupyter / Colab	Interface em células; o Colab roda na nuvem, sem instalar nada	Exploração de dados, aulas, prototipação

3.2 Anatomia do VS Code

- **Barra lateral:** Explorer (arquivos), Search, Source Control (Git), Run and Debug e Extensions.
- **Editor:** a área central onde o código é escrito. Aceita várias abas e divisão de tela.
- **Painel inferior:** Terminal, Problems (erros detectados), Output e Debug Console.
- **Barra de status:** mostra o interpretador Python e a versão em uso. Clique para trocar.

- **Paleta de comandos:** `Ctrl+Shift+P`. Quase tudo no VS Code é acessível digitando aqui.

3.3 Montando o ambiente passo a passo

1. Instale o **Python 3.11 ou superior** (python.org). No Windows, marque a opção *Add Python to PATH*.
2. Instale o **VS Code** (code.visualstudio.com).
3. Instale as extensões **Python** e **Jupyter** (ambas da Microsoft) na aba Extensions.
4. Crie uma pasta para o projeto e abra com *File > Open Folder*.
5. Crie um **ambiente virtual** no terminal (comandos abaixo). O **terminal** é a janela onde digitamos comandos em texto.
6. Selecione o interpretador: `Ctrl+Shift+P`, depois *Python: Select Interpreter*, e escolha o `.venv`.
7. Crie um arquivo `.py` ou `.ipynb` e execute.

```
# criar o ambiente virtual (dentro da pasta do projeto)
python -m venv .venv

# ativar no Windows (PowerShell)
.venv\Scripts\Activate.ps1

# ativar no Linux ou macOS
source .venv/bin/activate

# instalar uma biblioteca dentro do ambiente
pip install pandas

# registrar as dependências do projeto
pip freeze > requirements.txt
```

Na engenharia de dados. O ambiente virtual isola as dependências de cada projeto. Sem ele, duas pipelines que precisam de versões diferentes da mesma biblioteca quebram uma à outra. O arquivo `requirements.txt` torna o ambiente reproduzível em outra máquina ou em produção.

Para confirmar que o VS Code está usando o interpretador certo, rode a célula abaixo. `sys.version` mostra a versão do Python e `sys.executable` mostra o caminho do Python que está rodando. O caminho deve apontar para a pasta `.venv` do seu projeto.

```
import sys
print(sys.version)
print(sys.executable)
```

3.4 Atalhos que economizam tempo

Atalho	Função
`Ctrl+Shift+P`	Paleta de comandos
`Shift+Enter`	Executa a célula do notebook e avança para a próxima
`Ctrl+Enter`	Executa a célula e permanece nela
`Ctrl+/`	Comenta ou descomenta as linhas selecionadas
`Alt+seta acima` ou `abaixo`	Move a linha
`F5`	Inicia a depuração
`Ctrl+crase`	Abre e fecha o terminal (a crase fica acima do Tab)

No macOS, troque `Ctrl` por `Cmd`.

3.5 Script (.py) ou notebook (.ipynb)?

Um **script** é um arquivo de texto executado de ponta a ponta. Um **notebook** mistura texto, código e resultados em células que você executa uma a uma. Notebooks são ótimos para explorar dados e ensinar. Pipelines em produção costumam rodar como scripts `.py`, que são mais fáceis de versionar, testar e agendar.

4. Características do Python

Para conhecer uma linguagem, os profissionais costumam classificá-la respondendo algumas perguntas. Neste módulo respondemos cada uma para o Python, uma de cada vez, partindo do zero:

1. É de **alto** ou de **baixo nível**?
2. É **compilada** ou **interpretada**?
3. Tem tipagem **dinâmica** ou **estática**? **Forte** ou **fraca**?
4. É **imperativa** ou **declarativa**?

4.1 Alto nível ou baixo nível?

Primeiro, um conceito: a **memória**. Quando o computador trabalha, guarda os dados temporariamente na memória (a RAM). Imagine uma enorme estante com milhões de gavetas numeradas. O número é o **endereço** da gaveta. Para guardar um dado, é preciso escolher gavetas livres, usá-las e, no fim, liberá-las.

Nível diz o quanto a linguagem está perto da máquina ou perto do jeito humano de pensar.

Baixo nível significa perto da máquina. Você mesmo cuida de detalhes como reservar espaço na memória, dizer de quanto espaço precisa e devolvê-lo ao final. Veja como seria, em C (uma linguagem de baixo nível), guardar três números inteiros:

```
int *numeros = malloc(3 * sizeof(int));   // 1. reserva espaço na memória
numeros[0] = 10;                          // 2. guarda os valores
numeros[1] = 20;
numeros[2] = 30;
free(numeros);                            // 3. devolve o espaço
```

- **Linha 1:** `malloc` pede ao sistema um pedaço de memória, e **você** calcula o tamanho (3 números vezes o tamanho de cada inteiro).
- **Linhas 2 a 4:** você guarda os valores nas posições reservadas.
- **Linha 5:** `free` devolve o espaço. Se você esquecer, a memória fica ocupada sem necessidade (um *vazamento de memória*). Se usar uma posição errada, o programa pode travar ou corromper dados.

Em troca desse trabalho, você tem controle total e programas muito rápidos.

Alto nível significa perto do jeito humano de pensar. A linguagem esconde esses detalhes e cuida da memória por você. O mesmo exemplo em Python:

```
numeros = [10, 20, 30]
print(numeros)
```

SAÍDA

```
[10, 20, 30]
```

Uma linha. Você não reserva nem libera nada: o Python decide onde guardar e limpa sozinho quando o dado não é mais usado (esse faxineiro automático se chama *coletor de lixo*).

Uma analogia: dirigir um carro de **câmbio manual** (baixo nível) exige controlar a embreagem e trocar as marchas. Dá mais controle, mas dá mais trabalho e é fácil errar. O **câmbio automático** (alto nível) cuida disso por você, e você só pensa no destino.

Nível	Exemplos	Você cuida da memória?
Muito baixo	Assembly (instruções do processador, uma a uma)	Sim, de tudo
Baixo	C	Sim, reserva e libera
Alto	Python, Java, JavaScript	Não, a linguagem cuida

Na engenharia de dados. O preço do alto nível é velocidade: um laço puro em Python é bem mais lento que o equivalente em C. Isso raramente pesa, porque bibliotecas como NumPy e pandas, e motores como o Spark, executam o trabalho pesado em código de baixo nível otimizado (C, C++, Rust, Java), enquanto você escreve em Python. **Python funciona como a cola**: descreve o que fazer e delega a execução para motores rápidos.

4.2 Compilada ou interpretada?

Lembra do tradutor do Módulo 1? Existem dois tipos:

- **Compilador:** traduz o programa **inteiro** de uma vez, antes de executar, e gera um arquivo executável. É como traduzir um livro inteiro antes de entregá-lo ao leitor.
- **Interpretador:** lê e executa o código **aos poucos**, linha a linha. É como um intérprete que traduz uma conversa em tempo real.

C é tipicamente compilada. **Python é interpretada**: você escreve e executa direto, sem etapa extra de compilação. Isso é ótimo para testar e aprender, e é por isso que o notebook funciona célula a célula.

Curiosidade. Por dentro, o Python converte o seu código para um formato intermediário (o *bytecode*) e o executa numa máquina virtual. Para quem está começando basta saber: você escreve e roda.

4.3 Tipagem: dinâmica e forte

O **tipo** de um dado é a sua natureza: número inteiro, número decimal, texto e assim por diante (Módulo 6 detalha). A tipagem descreve como a linguagem lida com isso. São duas perguntas diferentes.

Dinâmica ou estática? Em linguagens estáticas, como C, você precisa declarar o tipo antes de usar (`int idade = 29;`). Em linguagens **dinâmicas**, como Python, não declara nada: o Python descobre o tipo pelo valor, e o mesmo nome pode apontar para valores de tipos diferentes ao longo do tempo. A função `type()` mostra o tipo de um valor.

```
x = 10
print(type(x))
```

SAÍDA

```
<class 'int'>
```

`int` significa número inteiro. Agora o mesmo nome `x` recebe um texto:

```
x = "dez"
print(type(x))
```

SAÍDA

```
<class 'str'>
```

`str` (de *string*) significa texto. O mesmo `x` mudou de tipo, e o Python aceitou sem reclamar. Isso é tipagem dinâmica.

Forte ou fraca? Em linguagem de tipagem **forte**, como Python, os tipos não se misturam em silêncio: somar um texto com um número gera um erro, em vez de a linguagem adivinhar o que você quis. Para mostrar o erro sem interromper o notebook, usamos `try` e `except`, que significam "tente executar isto; se der erro, faça aquilo". Você verá isso com calma no Módulo 10.

```
try:
    resultado = "5" + 2
except TypeError as erro:
    print("Erro de tipo:", erro)
```

SAÍDA

```
Erro de tipo: can only concatenate str (not "int") to str
```

A mensagem diz que só dá para juntar texto com texto. Em JavaScript, uma linguagem de tipagem fraca, `"5" + 2` devolveria `"52"` sem avisar nada.

4.4 Imperativa ou declarativa?

Chamamos de **paradigma** o jeito de pensar e organizar o código. Dois paradigmas importantes para quem trabalha com dados são o imperativo e o declarativo. Uma analogia ajuda.

Imagine que você quer comer uma pizza. Uma opção é ir à cozinha e seguir uma receita: "abra a massa, espalhe o molho, ponha o queijo, leve ao forno por 15 minutos". Você descreve **como** fazer, passo a passo. A outra opção é ligar para uma pizzaria e dizer: "quero uma pizza de calabresa". Você descreve **o que** quer, e a cozinha decide como fazer.

- **Imperativo** é a receita: você ordena cada passo, na ordem. Descreve o *como*.
- **Declarativo** é o pedido: você declara o resultado que quer. Descreve o *quê*.

Vamos ver isso com um problema de dados: **somar apenas as vendas acima de 100**, numa lista de vendas com os valores 50, 120, 300, 80 e 150. A resposta esperada é 120 + 300 + 150 = 570.

Estilo declarativo, em SQL. Pedimos o resultado, sem dizer como chegar nele:

```
SELECT SUM(valor) FROM vendas WHERE valor > 100;
```

Leia em português: "some o valor, da tabela vendas, onde o valor for maior que 100". Não dissemos como percorrer a tabela nem em que ordem. O banco de dados decide.

Estilo imperativo, em Python. Aqui escrevemos a receita. Em português, seria:

```
1. Comece com total igual a zero
2. Para cada valor da lista
3.     se o valor for maior que 100
4.         some o valor ao total
5. No fim, mostre o total
```

Primeiro, guardamos a lista de valores:

```
valores = [50, 120, 300, 80, 150]
```

Agora a receita em código. Você ainda não estudou `for` e `if` (são da Parte 2), então não precisa dominar: apenas compare com os passos acima. `for v in valores` significa “para cada `v` na lista `valores`”. `if v > 100` significa “se `v` for maior que 100”. O recuo (os espaços à esquerda) indica o que está dentro de cada passo.

```
total = 0
for v in valores:
    if v > 100:
        total = total + v
print("Imperativo:", total)
```

SAÍDA

```
Imperativo: 570
```

Python também permite escrever o mesmo de forma declarativa, em uma linha. Lê-se: “some `v`, para cada `v` em `valores`, se `v` for maior que 100”. Descrevemos o que queremos, sem a receita passo a passo.

```
print("Declarativo:", sum(v for v in valores if v > 100))
```

SAÍDA

```
Declarativo: 570
```

Mesmo resultado, dois estilos. Python é **multiparadigma**: aceita os dois (e outros, como orientação a objetos e programação funcional). Na prática, a maior parte do código Python é **imperativa**: você manda, passo a passo, o que o programa deve fazer.

> **Na engenharia de dados.** SQL e Spark são declarativos de verdade: você descreve a transformação e o motor decide como executá-la da forma mais eficiente. O pandas fica no meio do caminho: você descreve operações sobre colunas inteiras, sem escrever o laço. Já o Python imperativo faz a orquestração da pipeline: chama a API, lê o arquivo, trata o erro, dispara o próximo passo.

4.5 Resumo do módulo

Pergunta	Resposta para o Python
Nível	Alto nível: a linguagem cuida da memória
Compilada ou interpretada	Interpretada: escreve e executa direto
Tipagem	Dinâmica (não declara o tipo) e forte (não mistura tipos em silêncio)
Paradigma	Multiparadigma, predominantemente imperativo
Legibilidade	Sintaxe enxuta e indentação obrigatória deixam o código parecido com texto
Ecossistema	Centenas de milhares de pacotes no PyPI, instaláveis com `pip`

5. Variáveis

Uma variável é um **nome que referencia um valor** guardado na memória. Pense numa etiqueta colada no valor, e não numa caixa que guarda o valor dentro. Essa diferença explica comportamentos importantes mais adiante.

O sinal `=` é a **atribuição**. Ele não significa "igual" como na matemática: lê-se "o nome à esquerda passa a apontar para o valor à direita". A célula abaixo cria quatro variáveis, cada uma com um tipo diferente de valor: um texto (entre aspas), um número inteiro, um decimal (com ponto) e um valor lógico (`True` ou `False`, sempre com a primeira letra maiúscula).

```
nome_tabela = "clientes"
qtd_linhas = 1500
taxa_nulos = 0.032
ativa = True
```

Para ver o conteúdo das variáveis, usamos `print`. Quando damos vários valores separados por vírgula, o `print` os mostra separados por espaço.

```
print(nome_tabela, qtd_linhas, taxa_nulos, ativa)
```

SAÍDA

```
clientes 1500 0.032 True
```

5.1 Regras e convenções de nomes

- Pode conter letras, números e sublinhado (_), mas **não pode começar com número**.
- **Diferencia maiúsculas de minúsculas:** `total` e `Total` são nomes diferentes.
- **Não pode ser palavra reservada** (`class`, `for`, `if`, `None`...).
- Não pode ter espaço nem hífen.
- Padrão PEP 8: **snake_case** (`qtd_linhas`) para variáveis e MAIÚSCULAS para constantes (`LIMITE_MAXIMO`).
- Prefira nomes descritivos: `qtd_linhas` comunica mais que `x`.
- Não use nomes de funções prontas (`list`, `sum`, `max`, `str`) como variável, pois você as perde no resto do código.

Nome	Vale?	Motivo
`qtd_linhas`	Sim	Letras e sublinhado
`total2`	Sim	Número no fim é permitido
`2total`	Não	Começa com número
`nome-tabela`	Não	Hífen não é permitido (o Python lê como subtração)
`valor total`	Não	Espaço não é permitido
`class`	Não	Palavra reservada

5.2 Reatribuição e atribuição múltipla

Uma variável pode receber um novo valor a qualquer momento. A etiqueta simplesmente passa a apontar para outro valor, e o antigo é descartado.

```
contador = 1
print(contador)

contador = 2
print(contador)
```

SAÍDA

```
1
2
```

Python permite atribuir vários nomes de uma vez. A linha `a, b = 10, 20` dá 10 para `a` e 20 para `b`. Isso permite **trocar valores** sem variável auxiliar: `a, b = b, a`.

```
a, b = 10, 20
print(a, b)

a, b = b, a
print(a, b)
```

SAÍDA

```
10 20
20 10
```

Também dá para colocar duas etiquetas no mesmo valor com `x = y = 0`.

```
x = y = 0
print(x, y)
```

SAÍDA

```
0 0
```

5.3 Variável é referência

Como a variável é uma etiqueta, duas etiquetas podem apontar para o mesmo objeto. Quando o objeto é **mutável** (pode ser alterado no lugar, como uma lista), mudar por uma etiqueta muda o que a outra enxerga. Essa é uma fonte clássica de bugs em tratamento de dados. Vamos ver em três passos.

Passo 1. Criamos uma lista e, depois, `lista_b = lista_a`. Isso **não copia** a lista: apenas cria uma segunda etiqueta para a mesma lista.

```
lista_a = [1, 2, 3]
lista_b = lista_a
```

Passo 2. O método `.append(4)` adiciona o valor 4 no fim da lista. Mexemos pela etiqueta `lista_b`, mas olhe o que acontece com `lista_a`. O operador `is` pergunta se os dois nomes apontam para o mesmo objeto.

```
lista_b.append(4)

print(lista_a)
print(lista_a is lista_b)
```

SAÍDA

```
[1, 2, 3, 4]
True
```

Passo 3. Para ter uma cópia de verdade, usamos `.copy()`. Agora `lista_c` é uma lista independente.

```
lista_c = lista_a.copy()
lista_c.append(5)

print(lista_a)
print(lista_c)
```

SAÍDA

```
[1, 2, 3, 4]
[1, 2, 3, 4, 5]
```

6. Tipos de dados

Todo valor em Python tem um **tipo**, que define o que se pode fazer com ele. Por exemplo, dá para somar dois números, mas não faz sentido dividir um texto por outro. Os tipos básicos:

Tipo	Exemplo	Uso em dados
`int`	`42`	Identificadores, contagens, quantidades
`float`	`3.14`	Medidas, taxas, valores com casas decimais
`str`	`"Ana"`	Nomes, códigos, datas em formato texto
`bool`	`True`, `False`	Indicadores (ativo, válido, duplicado)
`NoneType`	`None`	Ausência de valor (o nulo)

6.1 Descobrindo o tipo de um valor

A função `type()` diz o tipo de qualquer valor. A função `isinstance(valor, tipo)` pergunta se o valor é daquele tipo e responde `True` ou `False`.

```
print(type(42))
print(type(3.5))
print(type("a"))
print(type(True))
```

SAÍDA

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

```
print(isinstance(3.5, float))
print(isinstance(3.5, int))
```

SAÍDA

```
True
False
```

6.2 Números: int e float

O `int` é o número inteiro, sem casas decimais, e **não tem limite de tamanho**. O operador `**` é a potência. Veja um número enorme:

```
print(2 ** 100)
```

SAÍDA

```
1267650600228229401496703205376
```

Para facilitar a leitura de números grandes, o Python aceita o sublinhado dentro do número:

```
print(1_000_000)
```

SAÍDA

```
1000000
```

O `float` é o número com casas decimais. Ele tem uma pegadinha. O computador guarda números em **binário** (zeros e uns), e alguns decimais, como 0,1, não têm representação exata em binário, do mesmo jeito que 1/3 não tem representação exata em decimal (0,333...). O resultado é uma pequena imprecisão:

```
print(0.1 + 0.2)
```

SAÍDA

```
0.30000000000000004
```

Para dinheiro e outros casos em que a exatidão importa, existe o tipo `Decimal`, que precisa ser trazido com `import`. Note que o valor entra como texto, entre aspas:

```
from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2"))
```

SAÍDA

```
0.3
```

A função `round(número, casas)` arredonda para a quantidade de casas indicada:

```
print(round(3.14159, 2))
```

SAÍDA

```
3.14
```

Atenção. Em dados financeiros, nunca compare valores `float` com `==` nem some centavos em `float` sem cuidado. Use `Decimal` ou trabalhe em centavos como `int`.

6.3 Strings (texto)

Uma string é uma sequência de caracteres entre aspas simples ou duplas. Para textos de várias linhas usam-se aspas triplas. Strings são **imutáveis**: nenhum método altera o texto original, todos devolvem um texto novo.

```
texto = "  Engenharia de Dados  "
print(texto)
```

SAÍDA

```
  Engenharia de Dados
```

Um **método** é uma função que pertence a um tipo e é chamada com um ponto depois do valor. `.upper()` devolve o texto em maiúsculas, `.lower()` em minúsculas e `.strip()` remove os espaços das pontas. Note que `texto` continua igual: guardamos o resultado de `.strip()` numa variável nova.

```
print(texto.upper())
print(texto.lower())
limpo = texto.strip()
print(limpo)
```

SAÍDA

```
  ENGENHARIA DE DADOS
  engenharia de dados
Engenharia de Dados
```

`len()` conta quantos caracteres o texto tem (espaços contam).

```
print(len(limpo))
```

SAÍDA

```
19
```

Indexação. Cada caractere ocupa uma posição, chamada de **índice**. A contagem começa em **0**, e índices negativos contam a partir do fim. Veja a palavra `Dados`:

```
 D   a   d   o   s
 0   1   2   3   4      (índices positivos: começam em 0)
-5  -4  -3  -2  -1      (índices negativos: contam do fim)
```

```
palavra = "Dados"
print(palavra[0])
print(palavra[-1])
```

SAÍDA

```
D
s
```

Fatiamento. `[início:fim]` extrai um pedaço do texto. O índice do início **entra**, o do fim **não entra**. Por isso `[1:4]` pega as posições 1, 2 e 3. Se omitir um dos lados, vai até a ponta.

```
print(palavra[1:4])
print(palavra[:2])
print(palavra[2:])
```

SAÍDA

```
ado
Da
dos
```

Outros métodos muito usados: `.replace(antigo, novo)` troca um trecho por outro, `.startswith(trecho)` pergunta se o texto começa com aquele trecho, e o operador `in` pergunta se um trecho está dentro do texto.

```
print(limpo.replace("Dados", "IA"))
print(limpo.startswith("Eng"))
print("Dados" in limpo)
```

SAÍDA

```
Engenharia de IA
True
True
```

`.split(separador)` quebra um texto em pedaços e devolve uma lista. `.join()` faz o caminho inverso. Os dois são a base para tratar linhas de arquivos como CSV, em que os campos vêm separados por um símbolo (aqui, o ponto e vírgula).

```
linha = "101;Ana;São Paulo;1500.50"
campos = linha.split(";")
print(campos)
```

SAÍDA

```
['101', 'Ana', 'São Paulo', '1500.50']
```

```
print(" | ".join(campos))
```

SAÍDA

```
101 | Ana | São Paulo | 1500.50
```

As **f-strings** (texto com `f` antes das aspas) inserem valores dentro do texto e controlam a formatação. Dentro das chaves, `:.1%` mostra como porcentagem com uma casa, e `:.2f` mostra duas casas decimais.

```
tabela = "vendas"
linhas = 12500
nulos = 0.0456
print(f"A tabela {tabela} tem {linhas} linhas e {nulos:.1%} de nulos.")
print(f"Valor: {1234.5:.2f}")
```

SAÍDA

```
A tabela vendas tem 12500 linhas e 4.6% de nulos.
Valor: 1234.50
```

6.4 Booleanos e valores “verdadeiros”

O tipo `bool` só tem dois valores: `True` e `False`. Além disso, em Python qualquer valor pode ser avaliado como verdadeiro ou falso com `bool()`. São considerados **falsos**: o número zero, o texto vazio, a coleção vazia e `None`. Todo o resto é verdadeiro.

```
print(bool(0), bool(""), bool([]), bool(None))
print(bool(1), bool("a"), bool([0]))
```

SAÍDA

```
False False False False
True True True
```

Cuidado com uma pegadinha: o texto `"False"` não está vazio, então é verdadeiro.

```
print(bool("False"))
```

SAÍDA

```
True
```

6.5 None: o nulo

`None` representa a **ausência de valor**. Não é zero e não é texto vazio. Em engenharia de dados é o equivalente ao `NULL` do SQL. Para testar, compara-se com `is None`.

```
resultado = None
print(resultado is None)
print(type(resultado))
print(resultado == 0, resultado == "")
```

SAÍDA

```
True
<class 'NoneType'>
False False
```

6.6 Convertendo entre tipos

As funções `int()`, `float()` e `str()` convertem um valor para outro tipo. Note que `int()` aplicado a um decimal **trunca** (corta as casas), não arredonda.

```
print(int("42") + 1)
print(float("3.5"))
print(str(42) + " itens")
print(int(3.99))
```

SAÍDA

```
43
3.5
42 itens
3
```

Quando a conversão é impossível, o Python gera um `ValueError`. Aqui usamos `try` e `except` de novo para mostrar a mensagem sem interromper o notebook.

```
try:
    int("abc")
except ValueError as erro:
    print("Erro:", erro)
```

SAÍDA

```
Erro: invalid literal for int() with base 10: 'abc'
```

Atenção. Dados que chegam de arquivos, formulários e APIs costumam vir como **texto**, mesmo quando parecem número. A função `input()`, que lê o que o usuário digita, também sempre devolve `str`. Converter para o tipo certo é uma das primeiras tarefas de qualquer pipeline.

6.7 Mutável ou imutável

Um valor **imutável** não pode ser alterado depois de criado (qualquer mudança cria um valor novo). Um **mutável** pode ser alterado no lugar. Essa classificação importa para o Módulo 8.

Imutáveis	Mutáveis
`int`, `float`, `str`, `bool`, `tuple`, `None`	`list`, `dict`, `set`

7. Operadores

Operadores são os símbolos que realizam operações sobre valores.

Grupo	Operadores	Observação
Aritméticos	`+ - * / // % **`	`/` sempre devolve `float`. `//` é divisão inteira. `%` é o resto
Comparação	`== != < > <= >=`	Resultado é `True` ou `False`
Lógicos	`and or not`	Combinam condições
Pertinência	`in not in`	Verifica se um item está numa sequência
Identidade	`is is not`	Verifica se é o mesmo objeto (use com `None`)
Atribuição composta	`+= -= *= /=`	`x += 1` equivale a `x = x + 1`

7.1 Aritméticos

Cada linha abaixo mostra o cálculo e o resultado. Observe as três divisões: `7 / 2` dá 3.5 (decimal), `7 // 2` dá 3 (só a parte inteira) e `7 % 2` dá 1 (o resto da divisão).

```
print("7 + 2  =", 7 + 2)
print("7 - 2  =", 7 - 2)
print("7 * 2  =", 7 * 2)
print("7 / 2  =", 7 / 2)
print("7 // 2 =", 7 // 2)
print("7 % 2  =", 7 % 2)
print("7 ** 2 =", 7 ** 2)
```

SAÍDA

```
7 + 2  = 9
7 - 2  = 5
7 * 2  = 14
7 / 2  = 3.5
7 // 2 = 3
7 % 2  = 1
7 ** 2 = 49
```

7.2 Comparação

Comparam dois valores e respondem `True` ou `False`. Atenção: `=` atribui, e `==` compara.

```
print(5 == 5.0)
print(5 != 6)
print(3 > 2)
print("a" < "b")
```

SAÍDA

```
True
True
True
True
```

7.3 Lógicos

`and` é verdadeiro só se **os dois** lados forem. `or` é verdadeiro se **pelo menos um** for. `not` inverte.

```
print(True and False)
print(True or False)
print(not True)
```

SAÍDA

```
False
True
False
```

7.4 Pertinência

`in` pergunta se um item está dentro de uma lista ou de um texto. `not in` pergunta o contrário.

```
print(3 in [1, 2, 3])
print("x" not in "abc")
```

SAÍDA

```
True
True
```

7.5 Atribuição composta

São atalhos para atualizar uma variável usando o próprio valor dela. `total += 50` é o mesmo que `total = total + 50`.

```
total = 100
total += 50
print(total)

total *= 2
print(total)
```

SAÍDA

```
150
300
```

Precedência. Python respeita a ordem: `**`, depois `* / // %`, depois `+ -`, depois comparações, depois `not`, `and`, `or`. Na dúvida, use parênteses. Eles não custam nada e evitam erros.

8. Coleções: list, tuple, dict e set

Dados raramente aparecem sozinhos. Uma **coleção** guarda vários valores sob um mesmo nome. Python tem quatro coleções principais:

Tipo	Sintaxe	Ordenado	Mutável	Duplicatas	Uso típico
`list`	`[1, 2, 3]`	Sim	Sim	Aceita	Sequência de registros
`tuple`	`(1, 2, 3)`	Sim	Não	Aceita	Registro fixo, chave composta
`dict`	`{"id": 1}`	Sim (inserção)	Sim	Chaves únicas	Registro, linha, JSON

Tipo	Sintaxe	Ordenado	Mutável	Duplicatas	Uso típico
`set`	`{1, 2, 3}`	Não	Sim	Não aceita	Valores distintos

8.1 list

Uma **lista** é uma sequência ordenada e mutável, entre colchetes. Os itens são acessados pelo índice, começando em 0, como nas strings.

```
ids = [101, 102, 103]
print(ids)
```

SAÍDA

```
[101, 102, 103]
```

Para alterar a lista: `.append(x)` adiciona no fim, `.insert(posição, x)` adiciona numa posição e `.remove(x)` remove pelo valor.

```
ids.append(104)
ids.insert(0, 100)
ids.remove(102)
print(ids)
```

SAÍDA

```
[100, 101, 103, 104]
```

Para consultar: `len()` conta os itens, `ids[0]` é o primeiro, `ids[-1]` é o último e `ids[1:3]` é uma fatia (posições 1 e 2).

```
print(len(ids))
print(ids[0], ids[-1])
print(ids[1:3])
```

SAÍDA

```
4
100 104
[101, 103]
```

`sorted()` devolve uma lista ordenada (sem alterar a original; `reverse=True` inverte). `sum()`, `min()` e `max()` calculam a soma, o menor e o maior.

```
print(sorted(ids, reverse=True))
print(sum(ids), min(ids), max(ids))
```

SAÍDA

```
[104, 103, 101, 100]
408 100 104
```

8.2 tuple

A **tupla** é parecida com a lista, mas **imutável**: depois de criada, não muda. Usa parênteses. É boa para dados que não devem mudar, como coordenadas. Dá para **desempacotar** seus valores em variáveis separadas numa única linha.

```
coordenada = (-19.92, -43.94)
lat, lon = coordenada
print(lat, lon)
```

SAÍDA

```
-19.92 -43.94
```

8.3 dict

O **dicionário** guarda pares **chave: valor**, entre chaves. Em vez de acessar por posição, você acessa pelo nome da chave. É a estrutura mais importante para quem trabalha com dados em Python, porque é a cara de um registro e de um JSON. Desde o Python 3.7 a ordem de inserção é mantida.

```
cliente = {"id": 1, "nome": "Ana", "cidade": "Belo Horizonte"}
print(cliente["nome"])
```

SAÍDA

```
Ana
```

Para adicionar uma chave nova ou alterar o valor de uma existente, usa-se a mesma sintaxe de atribuição:

```
cliente["email"] = "ana@exemplo.com"
cliente["cidade"] = "Contagem"
print(cliente)
```

SAÍDA

```
{'id': 1, 'nome': 'Ana', 'cidade': 'Contagem', 'email': 'ana@exemplo.com'}
```

Acessar uma chave que não existe com `cliente["telefone"]` gera `KeyError`. O método `.get(chave, padrão)` evita o erro e devolve o valor padrão. Isso é comum em dados vindos de APIs, em que campos podem faltar.

```
print(cliente.get("telefone", "não informado"))
```

SAÍDA

```
não informado
```

`.keys()` lista as chaves e `.values()` lista os valores. Envolvemos em `list()` para exibir como lista.

```
print(list(cliente.keys()))
print(list(cliente.values()))
```

SAÍDA

```
['id', 'nome', 'cidade', 'email']
[1, 'Ana', 'Contagem', 'ana@exemplo.com']
```

8.4 set

O **conjunto** guarda valores **únicos**, sem ordem. Ao transformar uma lista em `set`, as duplicatas somem automaticamente.

```
cidades = ["BH", "SP", "BH", "RJ", "SP"]
unicas = set(cidades)
print(len(unicas))
print(sorted(unicas))
```

SAÍDA

```
3
['BH', 'RJ', 'SP']
```

Conjuntos permitem as operações de conjuntos da matemática: `|` é a união (todos os itens), `&` é a interseção (só os que estão nos dois) e `-` é a diferença (os que estão no primeiro e não no segundo). Usamos `sorted()` só para exibir em ordem.

```
a = {1, 2, 3}
b = {3, 4}
print(sorted(a | b))
print(sorted(a & b))
print(sorted(a - b))
```

SAÍDA

```
[1, 2, 3, 4]
[3]
[1, 2]
```

Na engenharia de dados. `dict` é um registro ou um JSON. Uma `list` de `dict` é uma tabela ou a resposta de uma API. `set` é o `DISTINCT` do SQL. `tuple` serve de chave composta. Entender essa correspondência é o que torna pandas e PySpark naturais.

9. Mini projeto: relatório de vendas

Vamos juntar tudo: uma lista de dicionários (nossa tabela), conversões de tipo, coleções, operadores e f-strings, para produzir um relatório. Cada passo é uma célula.

Passo 1: os dados

Cada venda é um dicionário, e a lista `vendas` reúne todas. Para aprender: `len(vendas)` conta os registros, `vendas[0]` é o primeiro e `vendas[-1]["cliente"]` pega o cliente do último.

```
vendas = [
    {"id": 1, "cliente": "Ana",   "produto": "Notebook", "valor": 4500.00},
    {"id": 2, "cliente": "Bruno", "produto": "Mouse",    "valor": 89.90},
    {"id": 3, "cliente": "Ana",   "produto": "Teclado",  "valor": 250.00},
    {"id": 4, "cliente": "Carla", "produto": "Monitor",  "valor": 1200.00},
    {"id": 5, "cliente": "Bruno", "produto": "Notebook", "valor": 5200.00},
    {"id": 6, "cliente": "Diego", "produto": "Mouse",    "valor": 79.90},
]
```

```
print(len(vendas))
print(vendas[0])
print(vendas[-1]["cliente"])
```

SAÍDA

```
6
{'id': 1, 'cliente': 'Ana', 'produto': 'Notebook', 'valor': 4500.0}
Diego
```

Passo 2: extraindo colunas

Para somar os valores, precisamos de uma lista só com eles. A linha `[v["valor"] for v in vendas]` lê-se “para cada venda `v` na lista, pegue o valor”. Usando chaves em vez de colchetes, `{v["cliente"] for v in vendas}`, o resultado é um `set`, ou seja, os clientes sem repetição.

```
valores = [v["valor"] for v in vendas]
clientes = {v["cliente"] for v in vendas}
```

```
print(valores)
print(sorted(clientes))
```

SAÍDA

```
[4500.0, 89.9, 250.0, 1200.0, 5200.0, 79.9]
['Ana', 'Bruno', 'Carla', 'Diego']
```

Prévia. Essa construção é chamada de *comprehension* e é o jeito Python de extrair uma coluna. Será explicada com calma na Parte 2.

Passo 3: métricas

Agora usamos as funções que já conhecemos: `sum`, `len`, `max` e `min`. O ticket médio é o faturamento dividido pela quantidade de vendas.

```
total = sum(valores)
ticket_medio = total / len(valores)
maior = max(valores)
menor = min(valores)

print(round(total, 2), round(ticket_medio, 2), maior, menor)
```

SAÍDA

```
11319.8 1886.63 5200.0 79.9
```

Passo 4: o relatório

O formato numérico padrão do Python usa vírgula para milhar e ponto para decimal (11,319.80). Para o padrão brasileiro (11.319,80), trocamos os símbolos. Como vamos repetir isso várias vezes, empacotamos o código numa **função**, usando `def`. Uma função é um bloco de código com nome, que podemos chamar quando quisermos. Aqui, `brl` recebe um valor e devolve o texto formatado. Esta célula apenas define a função, e por isso não mostra nada.

```
def brl(valor):
    texto = f"{valor:,.2f}"
    texto = texto.replace(",", "X").replace(".", ",").replace("X", ".")
    return f"R$ {texto}"
```

Prévia. Funções são tema da Parte 2. Por ora, basta entender que `brl(5200)` devolve `R$ 5.200,00`.

Agora o relatório. `"=" * 40` repete o caractere 40 vezes, para desenhar uma linha. `", ".join(...)` junta os nomes dos clientes separados por vírgula.

```
print("=" * 40)
print("RELATÓRIO DE VENDAS")
print("=" * 40)
print(f"Registros:          {len(vendas)}")
print(f"Faturamento total:  {brl(total)}")
print(f"Ticket médio:       {brl(ticket_medio)}")
print(f"Maior venda:        {brl(maior)}")
print(f"Menor venda:        {brl(menor)}")
print(f"Clientes distintos: {len(clientes)} ({', '.join(sorted(clientes))})")
```

SAÍDA

```
========================================
RELATÓRIO DE VENDAS
========================================
Registros:          6
Faturamento total:  R$ 11.319,80
Ticket médio:       R$ 1.886,63
Maior venda:        R$ 5.200,00
Menor venda:        R$ 79,90
Clientes distintos: 4 (Ana, Bruno, Carla, Diego)
```

Passo 5: dict é JSON

O módulo `json` converte dicionários em texto JSON. O argumento `indent=2` deixa o texto bem recuado, e `ensure_ascii=False` mantém os acentos.

```
import json

print(json.dumps(vendas[0], indent=2, ensure_ascii=False))
```

SAÍDA

```
{
  "id": 1,
  "cliente": "Ana",
  "produto": "Notebook",
  "valor": 4500.0
}
```

A semelhança não é coincidência. O formato JSON, usado por praticamente toda API, foi desenhado sobre a mesma ideia de chave e valor.

10. Erros e tratamento de erros

Erros fazem parte do trabalho. A mensagem diz o tipo do erro e a linha. Leia sempre de baixo para cima: a última linha é a causa.

Erro	Causa mais comum	Exemplo
`SyntaxError`	Parêntese, aspas ou dois-pontos faltando; palavra reservada como nome	`print("oi"`
`IndentationError`	Recuo inconsistente	Linha dentro de bloco sem 4 espaços
`NameError`	Variável que não existe (digitada errada ou célula não executada)	`print(totl)`
`TypeError`	Operação entre tipos incompatíveis	`"5" + 2`
`ValueError`	Tipo certo, valor inválido	`int("abc")`
`KeyError`	Chave inexistente em dict	`cliente["telefone"]`

Erro	Causa mais comum	Exemplo
`IndexError`	Posição fora da lista	`[1, 2, 3][5]`
`ZeroDivisionError`	Divisão por zero	`10 / 0`

Dica de notebook. `NameError` em notebook quase sempre significa que uma célula anterior não foi executada. Use *Run All* para rodar tudo na ordem.

10.1 Capturando erros com try e except

Até aqui, quando um erro acontece, o Python **interrompe tudo** e mostra a mensagem. Só que, em dados, erro é coisa esperada: um texto onde deveria haver número, um campo que não veio, uma divisão por zero. Em vez de deixar o programa parar, podemos **capturar o erro** e decidir o que fazer. É um plano B.

Uma analogia: o trapezista que treina com uma rede de segurança embaixo. Se ele cair, a rede segura e o treino continua. O `try` é o trapézio e o `except` é a rede.

```
try:
    # código que pode dar erro
except TipoDoErro:
    # o que fazer se esse erro acontecer
```

- **try** significa “tente”. O bloco recuado abaixo dele é executado normalmente.
- **except** significa “exceto se”. Se o erro do tipo indicado acontecer dentro do `try`, o Python pula o resto do `try` e executa o bloco do `except`.
- Depois disso, o programa **continua** na linha seguinte, sem ser interrompido.

Vamos converter um texto em número. Com `"abc"` a conversão falha com `ValueError`, mas o erro é capturado. Repare que a última linha, fora do `try`, executa mesmo assim:

```
valor_texto = "abc"

try:
    numero = int(valor_texto)
    print("Convertido:", numero)
except ValueError:
    print("Não foi possível converter:", valor_texto)

print("O programa continuou")
```

SAÍDA

```
Não foi possível converter: abc
O programa continuou
```

Agora com um texto que **pode** ser convertido. Nenhum erro acontece, então o `except` nem é executado:

```
valor_texto = "42"

try:
    numero = int(valor_texto)
    print("Convertido:", numero)
except ValueError:
    print("Não foi possível converter:", valor_texto)

print("O programa continuou")
```

SAÍDA

```
Convertido: 42
O programa continuou
```

10.2 Vendo a mensagem do erro

Acrescentando `as erro` depois do tipo, o Python guarda a mensagem do erro na variável `erro` (o nome é escolha sua). Assim podemos mostrá-la, registrá-la ou usá-la no relatório. Uma divisão por zero é o exemplo clássico:

```
try:
    resultado = 10 / 0
except ZeroDivisionError as erro:
    print("Erro:", erro)
```

SAÍDA

```
Erro: division by zero
```

10.3 Mais de um tipo de erro

Um mesmo `try` pode ter vários `except`, cada um para um tipo de erro e com uma resposta diferente. O Python executa só o primeiro que combinar. Aqui o dado existe, mas não é um número:

```
dados = {"id": 1, "valor": "abc"}

try:
    numero = float(dados["valor"])
except ValueError:
    print("Valor não numérico")
except KeyError:
    print("Chave inexistente")
```

SAÍDA

```
Valor não numérico
```

Pedindo uma chave que não existe, o erro agora é outro, e cai no segundo `except`:

```
try:
    numero = float(dados["preco"])
except ValueError:
    print("Valor não numérico")
except KeyError as erro:
    print("Chave inexistente:", erro)
```

SAÍDA

```
Chave inexistente: 'preco'
```

Atenção. Existe um `except Exception:` que captura **qualquer** erro. É tentador, mas perigoso: ele esconde bugs de verdade, como um nome digitado errado. Prefira sempre capturar o tipo específico de erro que você espera.

10.4 else e finally

Dois blocos opcionais completam a estrutura. O **else** executa **só se nenhum erro aconteceu** no `try`. O **finally** executa **sempre**, com erro ou sem erro, e serve para tarefas de limpeza, como fechar um arquivo ou uma conexão.

```
try:
    numero = int("42")
except ValueError:
    print("Conversão falhou")
else:
    print("Deu certo:", numero)
finally:
    print("Isto roda sempre")
```

SAÍDA

```
Deu certo: 42
Isto roda sempre
```

Troque o `"42"` por `"abc"` e execute de novo: o `else` deixa de rodar, o `except` entra em ação, e o `finally` continua rodando.

10.5 Valor padrão quando a conversão falha

Um padrão muito usado em dados: tentar converter e, se falhar, guardar `None` (o nulo) no lugar. O dado inválido não derruba o processamento, e fica marcado como ausente para ser tratado depois.

```
texto = "R$ 12,50"

try:
    valor = float(texto)
except ValueError:
    valor = None

print(valor)
```

SAÍDA

```
None
```

Na engenharia de dados. Pipelines recebem dados de fora: arquivos, formulários, APIs. O inesperado é a regra, não a exceção. Capturar o erro específico permite escolher a resposta: descartar o registro, usar um valor padrão, registrar o problema ou parar tudo de propósito. O que não se pode é deixar um único registro ruim derrubar o processamento inteiro, nem engolir o erro sem avisar ninguém.

11. Exercícios

Exercício 1: Variáveis e f-string. Crie as variáveis `nome_tabela = "vendas"`, `qtd_linhas = 1500` e `taxa_nulos = 0.032`. Imprima: `A tabela vendas tem 1500 linhas e 3.2% de nulos.`

Exercício 2: Conversão de tipos. O texto `valor_texto = "R$ 1.234,56"` veio de um arquivo no padrão brasileiro. Converta para `float` e imprima `1234.56`. Dica: `replace` e `strip`.

Exercício 3: Strings. Dado `email = " Maria.Silva@Empresa.COM "`, imprima o e-mail normalizado (sem espaços e em minúsculas) e, em seguida, apenas o domínio (o que vem depois do `@`).

Exercício 4: Coleções. Dada a lista `ids = [101, 102, 101, 103, 102, 104]`, imprima quantos ids distintos existem e a lista ordenada sem duplicatas.

Exercício 5: Dicionário. Crie `registro = {"id": 1, "nome": "Ana", "ativo": True}`. Adicione a chave `email`, e depois imprima o valor da chave `telefone`, com o texto `não informado` como padrão, sem causar erro.

Exercício 6: Capturando um erro. O texto `entrada = "R$ 12,50"` não pode ser convertido direto com `float()`. Escreva um `try` e `except` que tente a conversão e, se der `ValueError`, guarde `None` em `valor` e imprima `Entrada inválida`. Ao final, imprima `valor`. Depois repita com `entrada = "12.50"`.

Exercício 7: Divisão por zero. Com `total = 500` e `quantidade = 0`, calcule o ticket médio (`total / quantidade`). Use `try` e `except` para imprimir `Sem vendas no período` em vez de deixar o erro aparecer.

12. Resumo e próximos passos

Conceito	Em uma frase
Linguagem de programação	Linguagem formal para instruir o computador
Algoritmo	Sequência finita de passos: entrada, processamento, saída
IDE	Editor, execução, depuração e terminal no mesmo lugar
Alto nível	A linguagem cuida da memória por você
Interpretada	O código é lido e executado aos poucos, sem etapa de compilação
Imperativo e declarativo	Descrever o como, passo a passo, ou descrever o quê se quer
Variável	Nome que referencia um valor
Tipos básicos	`int`, `float`, `str`, `bool`, `None`
Coleções	`list`, `tuple`, `dict`, `set`
Mutável	`list`, `dict`, `set`. Os demais são imutáveis
`try` e `except`	Captura um erro e decide o que fazer, sem interromper o programa

Na Parte 2

- Decisão com `if`, `elif` e `else`
- Laços: `for`, `range`, `enumerate` e `while`

- List comprehension
- Funções, parâmetros e retorno
- Mini projeto: validação de vendas e relatório por cliente

Gabarito dos exercícios

Existe mais de um jeito correto de resolver cada exercício. Tente primeiro sozinho e compare depois.

Exercício 1: Variáveis e f-string

```
nome_tabela = "vendas"
qtd_linhas = 1500
taxa_nulos = 0.032
print(f"A tabela {nome_tabela} tem {qtd_linhas} linhas e {taxa_nulos:.1%} de nulos.")
```

SAÍDA

```
A tabela vendas tem 1500 linhas e 3.2% de nulos.
```

Exercício 2: Conversão de tipos

```
valor_texto = "R$ 1.234,56"
limpo = valor_texto.replace("R$", "").replace(".", "").replace(",", ".").strip()
valor = float(limpo)
print(valor)
```

SAÍDA

```
1234.56
```

Exercício 3: Strings

```
email = "  Maria.Silva@Empresa.COM "
normalizado = email.strip().lower()
print(normalizado)
print(normalizado.split("@")[1])
```

SAÍDA

```
maria.silva@empresa.com
empresa.com
```

Exercício 4: Coleções

```
ids = [101, 102, 101, 103, 102, 104]
distintos = set(ids)
print(len(distintos))
print(sorted(distintos))
```

SAÍDA

```
4
[101, 102, 103, 104]
```

Exercício 5: Dicionário

```
registro = {"id": 1, "nome": "Ana", "ativo": True}
registro["email"] = "ana@exemplo.com"
print(registro.get("telefone", "não informado"))
print(registro)
```

SAÍDA

```
não informado
{'id': 1, 'nome': 'Ana', 'ativo': True, 'email': 'ana@exemplo.com'}
```

Exercício 6: Capturando um erro

```
entrada = "R$ 12,50"

try:
    valor = float(entrada)
except ValueError:
    valor = None
    print("Entrada inválida")

print(valor)

entrada = "12.50"

try:
    valor = float(entrada)
except ValueError:
    valor = None
    print("Entrada inválida")

print(valor)
```

SAÍDA

```
Entrada inválida
None
12.5
```

Exercício 7: Divisão por zero

```
total = 500
quantidade = 0

try:
    ticket = total / quantidade
    print(ticket)
except ZeroDivisionError:
    print("Sem vendas no período")
```

SAÍDA

```
Sem vendas no período
```

Glossário

Termo	Significado
Algoritmo	Sequência finita e ordenada de passos que resolve um problema
Bytecode	Código intermediário gerado pelo Python antes da execução
Compilador	Programa que traduz todo o código para linguagem de máquina antes de executar
Interpretador	Programa que lê e executa o código aos poucos, traduzindo durante a execução
Memória (RAM)	Onde o computador guarda os dados enquanto trabalha
Alto nível	Linguagem próxima do raciocínio humano, que esconde detalhes da máquina
Baixo nível	Linguagem próxima da máquina, em que o programador cuida de detalhes como a memória
Imperativo	Estilo que descreve como resolver, passo a passo
Declarativo	Estilo que descreve o que se quer, deixando o como para o motor
Paradigma	Jeito de pensar e organizar o código
IDE	Ambiente de Desenvolvimento Integrado
Kernel	Processo que executa o código das células de um notebook
Ambiente virtual	Pasta isolada com o Python e as bibliotecas de um projeto
Variável	Nome que referencia um valor na memória
Tipo	Categoria do valor, que define as operações possíveis
Mutável	Que pode ser alterado depois de criado
Exceção (erro)	Situação inesperada que interrompe o programa, como dividir por zero
try e except	Estrutura que tenta executar um código e define o que fazer se der erro
Função	Bloco de código com nome, que executa uma tarefa quando chamado
Método	Função que pertence a um tipo e é chamada com ponto (`texto.upper()`)
f-string	Texto com valores inseridos entre chaves, prefixado por `f`
PEP 8	Guia oficial de estilo do código Python
JSON	Formato de texto para dados em chave e valor, usado por APIs